

Wazuh SOC Detection Lab — My First SOC Project

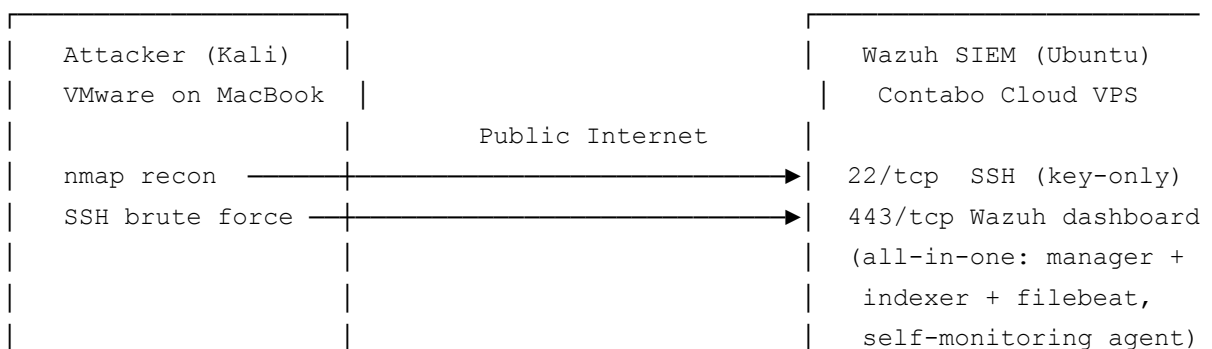
A cloud-hosted Wazuh SIEM I built from a bare Ubuntu server, hardened, and then attacked from a Kali machine over the public internet to test whether it would detect the intrusion. It did — and it also caught thousands of real, unsolicited attacks I never launched.

Overview

This build was my introduction into the SOC environment and cybersecurity as a whole. As I started applying for cybersecurity roles, I knew I needed experience, not just knowledge, and I figured the best place to learn was in my own environment — one I could test and get familiar with. It gave me hands-on time not only with the tools and dashboards I'd use as a SOC analyst or blue team member, but also with the tools that attackers and red teamers use to recon and exploit networks.

Architecture

I installed VMware on my computer to run Kali Linux, which from my research is typically the OS of choice for attackers. I got familiar with it by scanning my own router and home network to view open ports and check for vulnerabilities. After that came up fairly quiet, I set up a Wazuh SIEM on a VPS I already had, since my home lab isn't set up at the moment. I hardened the OS so nobody else could log in, got the IP addressing assigned, and confirmed the VPS was public and reachable. Then I began sending brute-force login attempts against the server to test detection. All attempts failed, because I'd hardened the system to key-only SSH, so it no longer accepts passwords.



```
different network
(home ISP, rotating IP)
```

```
UFW default-deny;
998/1000 ports filtere
```

A note on the dashboard: my home IP rotates about every seven days, which means I'll periodically lose dashboard access until I update the firewall rule by hand. My goal is to automate that (see **Future Work**) — forgetting it means getting locked out of my own monitoring.

Implementation

1. Harden the OS (SSH)

Created a non-root admin user, moved to key-only authentication, and disabled both root login and password login.

```
# On the server: create a non-root admin user and grant sudo
adduser reapr
usermod -aG sudo reapr

# On the Mac (the client I connect FROM): generate the key pair
ssh-keygen -t ed25519

# Push my public key to the server, then log in with the key
ssh-copy-id reapr@<VPS_PUBLIC_IP>
ssh reapr@<VPS_PUBLIC_IP>

# Lock SSH down
sudo nano /etc/ssh/sshd_config      # PermitRootLogin no
                                   # PasswordAuthentication no

sudo systemctl restart ssh
```

2. Install Wazuh (all-in-one)

Installed the Wazuh 4.14 all-in-one stack (manager + indexer + filebeat) on the VPS.

```
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh
sudo bash ./wazuh-install.sh -a
```

The all-in-one install self-monitors its own host, so it reports on the server it runs on without a separate agent. While exploring whether to add a *separate* agent, I hit a package conflict — `wazuh-agent` conflicts with `wazuh-manager` on the same box — which confirmed the

single all-in-one design already covers host monitoring. I kept the single-node design and attacked it directly.

3. Firewall (UFW)

Opened SSH to the internet (protected by key-only auth), pinned the dashboard to my home IP, and opened the Wazuh agent ports.

```
# Pull my current public IPv4 (the -4 matters — see Challenges & Lessons)
curl -4 ifconfig.me

sudo ufw allow 22/tcp
sudo ufw allow from <MY_HOME_IP> to any port 443 proto tcp
sudo ufw allow 1514/tcp
sudo ufw allow 1515/tcp
sudo ufw enable
sudo ufw status verbose
```

Resulting ruleset:

```
Status: active
Default: deny (incoming), allow (outgoing)

To          Action    From
--          -
22/tcp      ALLOW IN  Anywhere
443/tcp     ALLOW IN  <MY_HOME_IP>
1514/tcp    ALLOW IN  Anywhere
1515/tcp    ALLOW IN  Anywhere
```

The Detection

Ever since I was a kid watching the movie *Hackers*, I've had a vision of what hacking would look like. I knew the movies were dramatized, but that image stuck with me. I started with a basic nmap scan to look for open ports. Seeing that only 22 (SSH) and 443 (the dashboard) were open, while the other 998 showed as *filtered* — proof my firewall was silently dropping probes — I sent a series of brute-force login attempts against the server to see what the SIEM would detect. Spoiler: they all failed, thanks to the hardening from the initial setup.

Recon — from Kali, over the public internet:

```
nmap -sV <VPS_PUBLIC_IP>
```

```
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-11 07:12 -0400
Nmap scan report for <VPS_PUBLIC_IP>
Host is up (0.0088s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 8.9p1 Ubuntu 3ubuntu0.16 (protocol 2.0)
443/tcp   open  ssl/https
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Nmap done: 1 IP address (1 host up) scanned in 29.40 seconds
```

Three things this scan proves:

- **998 filtered ... (no-response)** — the firewall is working. *Filtered* means the packets were silently dropped with no reply at all, which is exactly what UFW's default-deny does. (That's different from *closed*, which would actively answer "nothing here.")
- **Only 22 and 443 are open** — my two intended doors, and nothing else.
- Even though nmap couldn't formally name the service on 443, the raw banner it returned leaked that it was my Wazuh dashboard (`osd-name: wazuh-siem` , a redirect to `/app/login` , and a `401 Unauthorized`). Being able to *read* what a scan returns — not just run it — is part of the job.

Screenshot: `screenshots/nmap-recon.png` — the full nmap output from Kali.

The payoff

This was genuinely one of the most eye-opening experiences of my entire IT and cybersecurity journey. I'd sent maybe 5 to 10 brute-force login attempts and one nmap scan. But when I opened the dashboard after the SIEM had been online about 8 hours, I had over 5,700 events logged — 5,363 authentication failures against just 26 successes. I was expecting maybe a few hundred. It hit me that almost none of those were mine — the vast majority was real automated attack traffic hammering my public IP from across the internet, which the SIEM had been quietly catching the whole time. The 26 successes were my own key-based logins. Being able to filter the logs — failed attempts, successful logins, account changes — and separate the handful of legitimate events from thousands of hostile ones showed me beyond words how much this work is needed.

The dashboard mapped the activity to specific Wazuh rules and MITRE ATT&CK techniques:

Rule	Description	MITRE ATT&CK

5712	SSHD brute force attempt	Brute Force
5710	Attempted login with non-existent user	Password Guessing / SSH

Screenshot: `screenshots/dashboard-overview.png` — 24h Threat Hunting view (~5,730 events, 5,363 auth failures, 26 successes, 0 level-12+). Screenshot: `screenshots/rule-5712.png` — SSHD brute-force detection. Screenshot: `screenshots/rule-5710.png` — non-existent-user detection. Screenshot: `screenshots/mitre-landscape.png` — full MITRE ATT&CK breakdown.

Analysis

Looking at the results over the roughly 8-hour period the server was up and reporting, it really showed how much activity hits an exposed host. I doubt anyone's sitting there manually — this is botnets and automated bots constantly mapping the internet, probing for anything vulnerable and reporting back "this is a target" or "this can be exploited." Some unsolicited internet traffic is benign — security scanners and researchers rather than attackers — so not all of it is malicious. But the events I was seeing were overwhelmingly failed SSH login attempts, which sits on the hostile end of that spectrum: something actively trying to get in. Separating that kind of noise from a genuine threat is exactly the triage a SOC analyst does day to day.

Challenges & Lessons

The biggest hurdle for me was one I've had since I first started with Ubuntu: learning the syntax and command structure of Linux itself. I've used a Mac throughout my IT journey and my associate's degree, so the Linux command line was less familiar. That showed up most when I was running two terminals at once — one SSH'd into the Linux server, the other running commands locally on my Mac. On a couple of occasions I got the two confused and ran a command in the wrong place, which tripped me up, but I was able to recognize it and recover each time.

A more specific challenge was whitelisting my IP in the firewall. I was trying to allow-list my address, but the rule kept getting rejected. The issue turned out to be that `curl ifconfig.me` was returning my IPv6 address, while UFW needed an IPv4 address for the rule. Once I understood that, I forced curl to return IPv4 with the `-4` flag, and the rule worked.

```
curl ifconfig.me          # returned an IPv6 address — UFW rejected the rule
curl -4 ifconfig.me       # forced IPv4 — the allow-list rule was accepted
```

Throughout the build, when I got stuck, I used the resources available to me — AI tooling and documentation — to diagnose the errors. The key for me wasn't just getting an answer; it was making sure I understood *why* each fix worked, like the IPv4-versus-IPv6 distinction, so I'd recognize it next time rather than just copying a solution.

Future Work

The next addition to this project will be automating the IP whitelist. Right now, if my ISP changes my home dynamic IP, I lose dashboard access until I manually update the UFW rule. I want to write a script that detects the IP change and updates the firewall rule automatically, removing that manual step and the risk of locking myself out.

After that, I'd like to build a script that analyzes the logs on a schedule and sorts them by category — attempted logins, successful logins, failed logins, and overall traffic patterns — then generates an automated readout emailed to me weekly or monthly. This would turn the raw event data into a digestible summary of what's been happening on the server over time, which is closer to the kind of reporting a security team relies on.

Further out, I'd like to expand into a true multi-host setup by adding a second machine running a Wazuh agent, and eventually bring a Windows endpoint with Sysmon into the monitoring, to broaden the environment beyond a single Linux host.

Stack

- **SIEM:** Wazuh 4.14 (all-in-one: manager + indexer + filebeat)
- **Server:** Ubuntu 22.04 LTS on a Contabo Cloud VPS
- **Attacker:** Kali Linux (VMware Fusion on macOS)
- **Firewall:** UFW (default-deny)
- **Recon:** nmap
- **Framework mapping:** MITRE ATT&CK

Scripts referenced in Future Work will live in [/scripts](#) as they're built.

